Nanoparticle Toxicity Prediction Using Data-Driven Methods

Complete Source Code Listing

Team Member	Enrolment No.
Pratham Joshi	KU2507U0361
Divyansh Khatri	KU2507U0188
Dhyey Bhatt	KU2507U0138
Jinay Shah	KU2507U0070

Code Bifurcation – which file does which job

The project is split so that each stage of the workflow lives in its own file. Nothing is duplicated: the web app and the command line tool, for example, both call the same prediction function rather than each having their own copy.

Stage	What it does	Workflow step	File
A	Configuration	Shared by every step	`src/config.py`
B	Dataset Collection	Step 3	`src/download_data.py`
C	Data Understanding and Raw Data Cleaning	Steps 4 and 5	`src/preprocess.py`
D	Exploratory Data Analysis and Feature Analysis	Steps 6 and 7	`src/eda.py`
E	Model Development, Training, Testing and Evaluation	Steps 8 to 13	`src/train_models.py`
F	Final Prediction System	Step 14	`src/predict.py` `app.py`
G	Pipeline Runner	Runs steps 3 to 13 in order	`run_all.py`
H	Bug Sweep Test Suite	Step 15	`tests/test_pipeline.py`

> **Order of execution.** `run_all.py` executes stages B → C → D → E in that order. Stage F is used afterwards, once a trained model exists. Stage H is run separately with `pytest`.

Supporting files

requirements.txt — 9 lines

```
# Nanoparticle Toxicity Prediction - project dependencies
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
matplotlib>=3.7
seaborn>=0.12
joblib>=1.3
streamlit>=1.28
```

dataset/README.txt — 59 lines

```
DATASET FILES
=============

Source : NanoTox metal-oxide nanoparticle dataset
Repo   : https://github.com/NanoTox/ToxicityModel  (file dataset.txt)
Paper  : NanoTox: Development of a parsimonious in silico model for toxicity
         assessment of metal-oxide nanoparticles using physicochemical features
         bioRxiv 2021, doi:10.1101/2021.02.22.432301


1_raw_nanotox_dataset.csv
    The original dataset exactly as downloaded, just converted from
    tab-separated to comma-separated so it opens in Excel.
    483 rows, 24 columns.
    NOTE: this file still contains the 'viability' column. Do NOT use
    that column as a model input - the toxic/nonToxic label was made
    from it, so using it leaks the answer.

2_cleaned_dataset_used_for_training.csv
    This is the file the model was actually trained on.
    355 rows, after:
      - removing 6 exact duplicate rows
```

```
      - dropping the leaky 'viability' column
      - merging 122 repeated experiments into one row each
        (majority vote on the label, ties count as toxic)
    Target column is 'toxic'  (1 = toxic, 0 = non-toxic).
    68 toxic / 287 non-toxic.
    'n_replicates' says how many raw rows were merged into that row.
    'NPs' and 'Cellline' are kept for reference but are NOT model inputs.

3_material_properties_lookup.csv
    The 13 chemistry descriptors that are constant for each material.
    The prediction app uses this so the user only has to enter the 6
    properties they can actually measure.


COLUMN MEANINGS
---------------
coresize    core size of the particle (nm)
hydrosize   hydrodynamic size in solution (nm)
surfcharge  surface charge / zeta potential (mV)
surfarea    surface area (m2/g)
Expotime    exposure time (hours)
dosage      dose given to the cells (ug/mL)
Celltype    Cancer or Normal
Hsf         heat of formation
Ec          conduction band energy
Ev          valence band energy
MeO         metal-oxygen bond descriptor
enthalpy    enthalpy
ratio       metal to oxygen ratio
e           electronegativity
esum        sum of electronegativity
esumbyo     electronegativity per oxygen
MW          molecular weight
NMetal      number of metal atoms
NOxygen     number of oxygen atoms
ox          oxidation state
```

A Configuration

Shared by every step

> Every path and column name lives in one file, so the rest of the project never hard-codes them. Changing the dataset mostly means editing only this file.

src/config.py — 92 lines

```python
"""
config.py
---------
All the paths and column names live here so we don't repeat them
in every script. If we ever change the dataset, we mostly only
edit this file.
"""

from pathlib import Path

# ---------- Folders ----------
PROJECT_DIR = Path(__file__).resolve().parent.parent

DATA_DIR = PROJECT_DIR / "data"
RAW_DIR = DATA_DIR / "raw"
PROCESSED_DIR = DATA_DIR / "processed"
MODEL_DIR = PROJECT_DIR / "models"
RESULTS_DIR = PROJECT_DIR / "results"
FIGURES_DIR = RESULTS_DIR / "figures"

# Files
RAW_FILE = RAW_DIR / "nanotox_dataset.tsv"
CLEAN_FILE = PROCESSED_DIR / "clean_dataset.csv"
MATERIAL_FILE = PROCESSED_DIR / "material_properties.csv"
BEST_MODEL_FILE = MODEL_DIR / "best_model.joblib"
METRICS_FILE = RESULTS_DIR / "model_comparison.csv"

# Where the dataset came from (see README)
DATA_URL = "https://raw.githubusercontent.com/NanoTox/ToxicityModel/master/dataset.txt"


# ---------- Columns ----------
TARGET_COL = "class"          # "Toxic" / "nonToxic"
TARGET_NUMERIC = "toxic"      # 1 / 0  (we make this in preprocess.py)

# The physicochemical + experimental properties we use as inputs.
# These are the ones the abstract talks about.
NUMERIC_FEATURES = [
    "coresize",       # core size of the particle (nm)
    "hydrosize",      # hydrodynamic size in solution (nm)
    "surfcharge",     # zeta potential / surface charge (mV)
    "surfarea",       # surface area (m2/g)
    "Expotime",       # exposure time (hours)
    "dosage",         # dose given to the cells (ug/mL)
    "Hsf",            # heat of formation
    "Ec",             # conduction band energy
    "Ev",             # valence band energy
    "MeO",            # metal-oxygen bond related descriptor
    "enthalpy",       # enthalpy
    "ratio",          # metal to oxygen ratio
    "e",              # electronegativity
    "esum",           # sum of electronegativity
    "esumbyo",        # electronegativity per oxygen
    "MW",             # molecular weight
    "NMetal",         # number of metal atoms
    "NOxygen",        # number of oxygen atoms
    "ox",             # oxidation state
]

CATEGORICAL_FEATURES = [
    "Celltype",       # Cancer / Normal
]

# IMPORTANT: 'viability' must NEVER be used as a feature.
```

```python
# The toxic / nonToxic label was created FROM the viability value,
# so keeping it would leak the answer and give a fake 100% accuracy.
LEAKY_COLS = ["viability"]

# 'NPs' (material name) and 'Cellline' are kept in the clean file only
# for plotting / grouping, not for training.
ID_COLS = ["NPs", "Cellline"]

# The 6 properties a user actually measures / chooses in the lab.
# These are the ones the prediction app asks for.
USER_INPUT_FEATURES = [
    "coresize",
    "hydrosize",
    "surfcharge",
    "surfarea",
    "Expotime",
    "dosage",
]

# The remaining descriptors are fixed for a given material
# (Al2O3 always has the same molecular weight, etc.), so the app
# looks them up from data/processed/material_properties.csv instead
# of asking the user.
MATERIAL_FEATURES = [c for c in NUMERIC_FEATURES if c not in USER_INPUT_FEATURES]

RANDOM_STATE = 42
TEST_SIZE = 0.25
```

B Dataset Collection

Step 3

> Downloads the NanoTox dataset if it is not already present, so the whole project can be rebuilt from an empty folder.

src/download_data.py — 39 lines

```python
"""
Step 3 - Dataset Collection
---------------------------
Downloads the nanoparticle toxicity dataset if it is not already
present in data/raw/.

Run:  python src/download_data.py
"""

import sys
import urllib.request

import config


def download():
    config.RAW_DIR.mkdir(parents=True, exist_ok=True)

    if config.RAW_FILE.exists():
        print(f"Dataset already present at {config.RAW_FILE}")
        print("Delete it first if you want a fresh copy.")
        return

    print(f"Downloading from {config.DATA_URL} ...")
    try:
        urllib.request.urlretrieve(config.DATA_URL, config.RAW_FILE)
    except Exception as err:
        print("Download failed:", err)
        print("You can also download the file manually and save it as:")
        print("  ", config.RAW_FILE)
        sys.exit(1)

    size_kb = config.RAW_FILE.stat().st_size / 1024
    print(f"Saved {config.RAW_FILE.name} ({size_kb:.1f} KB)")


if __name__ == "__main__":
    download()
```

C Data Understanding and Raw Data Cleaning

Steps 4 and 5

> describe_data() prints the report on the raw file. clean_data() does the cleaning: duplicate removal, target encoding, dropping the leaky viability column, type coercion and median filling. collapse_replicates() then merges repeated experiments into one row each, which is what takes the dataset from 477 rows to 355 unique experiments.

src/preprocess.py — 203 lines

```python
"""
Steps 4 & 5 - Data Understanding + Data Preprocessing
-----------------------------------------------------
Loads the raw tab separated file, prints a small report about it,
cleans it up and saves a tidy CSV in data/processed/.

Run:  python src/preprocess.py
"""

import pandas as pd

import config


def load_raw():
    """Read the raw dataset. It is tab separated, not comma separated."""
    if not config.RAW_FILE.exists():
        raise FileNotFoundError(
            f"{config.RAW_FILE} not found. Run 'python src/download_data.py' first."
        )
    return pd.read_csv(config.RAW_FILE, sep="\t")


def describe_data(df):
    """Step 4 - just look at the data and print what we find."""
    print("=" * 60)
    print("STEP 4 : DATA UNDERSTANDING")
    print("=" * 60)
    print(f"Rows: {df.shape[0]}   Columns: {df.shape[1]}")
    print("\nColumns and data types:")
    print(df.dtypes.to_string())

    print("\nMissing values per column:")
    missing = df.isna().sum()
    if missing.sum() == 0:
        print("  No missing values found.")
    else:
        print(missing[missing > 0].to_string())

    print(f"\nDuplicate rows: {df.duplicated().sum()}")

    print("\nTarget variable distribution:")
    counts = df[config.TARGET_COL].value_counts()
    for label, n in counts.items():
        print(f"  {label:10s} {n:4d}  ({n / len(df) * 100:.1f}%)")
    print("  -> The dataset is imbalanced, so accuracy alone is not enough.")

    if "NPs" in df.columns:
        print("\nNanoparticle materials in the dataset:")
        print(" ", ", ".join(sorted(df["NPs"].unique())))


def clean_data(df):
    """Step 5 - the actual cleaning."""
    print("\n" + "=" * 60)
    print("STEP 5 : DATA PREPROCESSING")
    print("=" * 60)

    before = len(df)
    df = df.drop_duplicates().reset_index(drop=True)
    print(f"Removed {before - len(df)} duplicate rows.")

```

```python
    # Make the target numeric: Toxic -> 1, nonToxic -> 0
    df[config.TARGET_NUMERIC] = (
        df[config.TARGET_COL].str.strip().str.lower().eq("toxic").astype(int)
    )
    print("Encoded target: Toxic = 1, nonToxic = 0")

    # Drop the leaky column. See the note in config.py.
    dropped = [c for c in config.LEAKY_COLS if c in df.columns]
    df = df.drop(columns=dropped)
    print(f"Dropped leaky column(s): {dropped}")

    # Numeric columns should really be numeric
    for col in config.NUMERIC_FEATURES:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    # Fill any missing numeric value with the median of that column.
    # (This dataset has none, but we keep the step so the pipeline is safe.)
    n_missing = df[config.NUMERIC_FEATURES].isna().sum().sum()
    if n_missing:
        df[config.NUMERIC_FEATURES] = df[config.NUMERIC_FEATURES].fillna(
            df[config.NUMERIC_FEATURES].median()
        )
        print(f"Filled {n_missing} missing numeric values with the column median.")
    else:
        print("No missing numeric values to fill.")

    # Tidy up the categorical column
    for col in config.CATEGORICAL_FEATURES:
        df[col] = df[col].astype(str).str.strip()

    keep = (
        config.ID_COLS
        + config.NUMERIC_FEATURES
        + config.CATEGORICAL_FEATURES
        + [config.TARGET_NUMERIC]
    )
    keep = [c for c in keep if c in df.columns]
    df = df[keep]

    df = collapse_replicates(df)

    print(f"Final clean shape: {df.shape}")
    return df


def collapse_replicates(df):
    """
    Merge repeated experiments into one row.

    We found this during the bug sweep. The raw file has the same
    experiment (same particle, same dose, same cell line) recorded
    several times with a slightly different viability each time.
    Once we drop the viability column those rows become identical.

    That is a problem because train_test_split would then put copies
    of the SAME experiment in both the training set and the test set,
    and the test score would look better than it really is.

    So we keep one row per unique set of properties. If the repeats
    disagree on the label we take the majority, and if it is an exact
    tie we call it toxic, because for a safety tool it is better to
    wrongly warn than to wrongly clear.
    """
    features = config.NUMERIC_FEATURES + config.CATEGORICAL_FEATURES

    n_dupes = df.duplicated(subset=features).sum()
    if n_dupes == 0:
        df["n_replicates"] = 1
        return df.reset_index(drop=True)

    grouped = df.groupby(features, dropna=False, sort=False)

    conflicting = (grouped[config.TARGET_NUMERIC].nunique() > 1).sum()

    merged = grouped.agg(
```

```
        NPs=("NPs", "first"),
        Cellline=("Cellline", "first"),
        n_replicates=(config.TARGET_NUMERIC, "size"),
        toxic_mean=(config.TARGET_NUMERIC, "mean"),
    ).reset_index()

    # majority vote, ties count as toxic
    merged[config.TARGET_NUMERIC] = (merged["toxic_mean"] >= 0.5).astype(int)
    merged = merged.drop(columns=["toxic_mean"])

    print(
        f"Merged {n_dupes} repeated experiments "
        f"({len(df)} rows -> {len(merged)} unique experiments)."
    )
    print(
        f"  {conflicting} of them had repeats that disagreed on the label "
        f"- majority vote used."
    )

    order = (
        config.ID_COLS
        + config.NUMERIC_FEATURES
        + config.CATEGORICAL_FEATURES
        + [config.TARGET_NUMERIC, "n_replicates"]
    )
    return merged[[c for c in order if c in merged.columns]].reset_index(drop=True)


def save_material_lookup(df):
    """
    Build a small lookup table of the descriptors that are fixed for
    a material (molecular weight, enthalpy, oxidation state ...).

    The prediction app uses this so the user only has to type the 6
    properties they can actually measure, and we fill in the chemistry
    for them.
    """
    lookup = df.groupby("NPs")[config.MATERIAL_FEATURES].first()

    # Sanity check: these really should be constant inside a material.
    varying = [
        c
        for c in config.MATERIAL_FEATURES
        if df.groupby("NPs")[c].nunique().max() > 1
    ]
    if varying:
        print(f"  warning: these are NOT constant per material: {varying}")

    lookup.to_csv(config.MATERIAL_FILE)
    print(f"Saved material lookup ({len(lookup)} materials) to {config.MATERIAL_FILE}")


def main():
    df = load_raw()
    describe_data(df)
    clean = clean_data(df)

    config.PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    clean.to_csv(config.CLEAN_FILE, index=False)
    print(f"\nSaved cleaned data to {config.CLEAN_FILE}")
    save_material_lookup(clean)


if __name__ == "__main__":
    main()
```

D Exploratory Data Analysis and Feature Analysis

Steps 6 and 7

> Produces all ten figures and ranks the features by their correlation with toxicity.

src/eda.py — 157 lines

```python
"""
Steps 6 & 7 - Exploratory Data Analysis + Feature Analysis
-----------------------------------------------------------
Makes the plots that go into the report and saves them in
results/figures/.

Run:  python src/eda.py
"""

import matplotlib

matplotlib.use("Agg")  # save plots to files instead of opening windows

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

import config

sns.set_theme(style="whitegrid")

# The properties the abstract specifically mentions - we plot these in
# detail. Same list the prediction app asks the user for.
MAIN_PROPS = config.USER_INPUT_FEATURES


def load_clean():
    if not config.CLEAN_FILE.exists():
        raise FileNotFoundError(
            f"{config.CLEAN_FILE} not found. Run 'python src/preprocess.py' first."
        )
    return pd.read_csv(config.CLEAN_FILE)


def save(fig, name):
    config.FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    path = config.FIGURES_DIR / name
    fig.savefig(path, dpi=120, bbox_inches="tight")
    plt.close(fig)
    print("  saved", path.name)


def plot_target_balance(df):
    fig, ax = plt.subplots(figsize=(5, 4))
    counts = df[config.TARGET_NUMERIC].value_counts().sort_index()
    ax.bar(["Non-Toxic", "Toxic"], counts.values, color=["#4c9f70", "#d1495b"])
    for i, v in enumerate(counts.values):
        ax.text(i, v + 4, str(v), ha="center")
    ax.set_ylabel("Number of samples")
    ax.set_title("Class balance in the dataset")
    save(fig, "01_class_balance.png")


def plot_histograms(df):
    fig, axes = plt.subplots(2, 3, figsize=(14, 8))
    for ax, col in zip(axes.ravel(), MAIN_PROPS):
        ax.hist(df[col], bins=25, color="#3f7cac", edgecolor="white")
        ax.set_title(col)
        ax.set_xlabel("")
    fig.suptitle("Distribution of the main physicochemical properties", fontsize=14)
    fig.tight_layout()
    save(fig, "02_histograms.png")


def plot_toxic_vs_nontoxic(df):
```

```python
    """Box plots comparing toxic and non-toxic groups."""
    fig, axes = plt.subplots(2, 3, figsize=(14, 8))
    tmp = df.copy()
    tmp["Toxicity"] = tmp[config.TARGET_NUMERIC].map({0: "Non-Toxic", 1: "Toxic"})
    for ax, col in zip(axes.ravel(), MAIN_PROPS):
        sns.boxplot(
            data=tmp, x="Toxicity", y=col, ax=ax,
            hue="Toxicity", palette=["#4c9f70", "#d1495b"], legend=False,
        )
        ax.set_title(col)
        ax.set_xlabel("")
    fig.suptitle("Toxic vs Non-Toxic comparison", fontsize=14)
    fig.tight_layout()
    save(fig, "03_toxic_vs_nontoxic.png")


def plot_scatter(df):
    tmp = df.copy()
    tmp["Toxicity"] = tmp[config.TARGET_NUMERIC].map({0: "Non-Toxic", 1: "Toxic"})
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    sns.scatterplot(
        data=tmp, x="coresize", y="hydrosize", hue="Toxicity",
        palette=["#4c9f70", "#d1495b"], alpha=0.75, ax=axes[0],
    )
    axes[0].set_title("Core size vs Hydrodynamic size")
    sns.scatterplot(
        data=tmp, x="dosage", y="surfcharge", hue="Toxicity",
        palette=["#4c9f70", "#d1495b"], alpha=0.75, ax=axes[1],
    )
    axes[1].set_title("Dosage vs Surface charge")
    fig.tight_layout()
    save(fig, "04_scatter_plots.png")


def plot_correlation(df):
    cols = config.NUMERIC_FEATURES + [config.TARGET_NUMERIC]
    corr = df[cols].corr()
    fig, ax = plt.subplots(figsize=(12, 10))
    sns.heatmap(corr, cmap="coolwarm", center=0, square=True,
                linewidths=0.4, cbar_kws={"shrink": 0.7}, ax=ax)
    ax.set_title("Correlation heatmap of all numeric features")
    save(fig, "05_correlation_heatmap.png")

    # Which features move the most with toxicity?
    print("\nCorrelation of each feature with toxicity (strongest first):")
    ranked = (
        corr[config.TARGET_NUMERIC]
        .drop(config.TARGET_NUMERIC)
        .sort_values(key=abs, ascending=False)
    )
    for name, value in ranked.items():
        print(f"  {name:12s} {value:+.3f}")
    return ranked


def plot_by_material(df):
    rate = (
        df.groupby("NPs")[config.TARGET_NUMERIC]
        .agg(["mean", "count"])
        .sort_values("mean", ascending=False)
    )
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(rate.index, rate["mean"] * 100, color="#e07a5f")
    ax.set_ylabel("% of samples that are toxic")
    ax.set_title("Toxicity rate by nanoparticle material")
    for i, (m, c) in enumerate(zip(rate["mean"], rate["count"])):
        ax.text(i, m * 100 + 1, f"n={c}", ha="center", fontsize=8)
    save(fig, "06_toxicity_by_material.png")
    print("\nToxicity rate per material:")
    print((rate["mean"] * 100).round(1).to_string())


def main():
    df = load_clean()
    print("Making EDA plots ...")
```

```
    plot_target_balance(df)
    plot_histograms(df)
    plot_toxic_vs_nontoxic(df)
    plot_scatter(df)
    ranked = plot_correlation(df)
    plot_by_material(df)

    config.RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    ranked.rename("correlation_with_toxicity").to_csv(
        config.RESULTS_DIR / "feature_correlation.csv"
    )
    print("\nAll figures are in", config.FIGURES_DIR)


if __name__ == "__main__":
    main()
```

E Model Development, Training, Testing and Evaluation

Steps 8 to 13

> Builds the five classifiers, trains them, evaluates them on the held-out test set, plots the confusion matrices and ROC curves, selects the best model by F1, saves it, and runs the leave-one-material-out generalisation check.

src/train_models.py — 371 lines

```python
"""
Steps 8 to 12 - Model Development, Training, Testing, Evaluation,
                and picking the best model
----------------------------------------------------------------
Trains the five classifiers listed in the abstract:
    Logistic Regression, Decision Tree, KNN, SVM, Random Forest
compares them, and saves the winner to models/best_model.joblib.

Run:  python src/train_models.py
"""

import matplotlib

matplotlib.use("Agg")

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import (
    LeaveOneGroupOut,
    StratifiedKFold,
    cross_val_score,
    train_test_split,
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

import config

sns.set_theme(style="whitegrid")


def build_preprocessor():
    """Scale the numbers, one-hot encode the Cell type column."""
    return ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), config.NUMERIC_FEATURES),
            (
                "cat",
                OneHotEncoder(handle_unknown="ignore"),
                config.CATEGORICAL_FEATURES,
            ),
        ]
    )

```

```python

def get_models():
    """
    The five classifiers mentioned in the abstract.

    class_weight = balanced is used because only about 16% of the
    samples are toxic. Without it the models just predict non-toxic
    for everything and still look 84% accurate, which is useless.
    """
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=2000,
            class_weight="balanced",
            random_state=config.RANDOM_STATE,
        ),
        "Decision Tree": DecisionTreeClassifier(
            max_depth=6,
            class_weight="balanced",
            random_state=config.RANDOM_STATE,
        ),
        "KNN": KNeighborsClassifier(n_neighbors=5),
        "SVM": SVC(
            kernel="rbf",
            probability=True,
            class_weight="balanced",
            random_state=config.RANDOM_STATE,
        ),
        "Random Forest": RandomForestClassifier(
            n_estimators=300,
            class_weight="balanced",
            random_state=config.RANDOM_STATE,
            n_jobs=-1,
        ),
    }


def make_pipeline(model):
    return Pipeline([("prep", build_preprocessor()), ("model", model)])


def evaluate(name, pipe, X_test, y_test):
    """Step 11 - all the metrics for one model."""
    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1]
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_test, y_prob),
    }


def plot_confusion_matrices(fitted, X_test, y_test):
    n = len(fitted)
    fig, axes = plt.subplots(1, n, figsize=(4 * n, 4))
    axes = np.atleast_1d(axes)
    for ax, (name, pipe) in zip(axes, fitted.items()):
        cm = confusion_matrix(y_test, pipe.predict(X_test))
        ConfusionMatrixDisplay(
            cm, display_labels=["Non-Toxic", "Toxic"]
        ).plot(ax=ax, colorbar=False, cmap="Blues")
        ax.set_title(name, fontsize=10)
        ax.grid(False)
    fig.suptitle("Confusion matrices on the test set", fontsize=14)
    fig.tight_layout()
    path = config.FIGURES_DIR / "07_confusion_matrices.png"
    fig.savefig(path, dpi=120, bbox_inches="tight")
    plt.close(fig)
    print("  saved", path.name)


def plot_roc(fitted, X_test, y_test):
    fig, ax = plt.subplots(figsize=(6, 5))
```

```python
    for name, pipe in fitted.items():
        prob = pipe.predict_proba(X_test)[:, 1]
        fpr, tpr, _ = roc_curve(y_test, prob)
        ax.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test, prob):.3f})")
    ax.plot([0, 1], [0, 1], "k--", linewidth=1, label="Random guess")
    ax.set_xlabel("False positive rate")
    ax.set_ylabel("True positive rate")
    ax.set_title("ROC curves")
    ax.legend(fontsize=8)
    path = config.FIGURES_DIR / "08_roc_curves.png"
    fig.savefig(path, dpi=120, bbox_inches="tight")
    plt.close(fig)
    print("  saved", path.name)


def plot_scores(table):
    melted = table.melt(
        id_vars="Model",
        value_vars=["Accuracy", "Precision", "Recall", "F1"],
        var_name="Metric",
        value_name="Score",
    )
    fig, ax = plt.subplots(figsize=(10, 5))
    sns.barplot(data=melted, x="Model", y="Score", hue="Metric", ax=ax)
    ax.set_ylim(0, 1.05)
    ax.set_title("Model comparison on the test set")
    ax.tick_params(axis="x", rotation=15)
    path = config.FIGURES_DIR / "09_model_comparison.png"
    fig.savefig(path, dpi=120, bbox_inches="tight")
    plt.close(fig)
    print("  saved", path.name)


def plot_feature_importance(pipe, name):
    """Step 13 - which properties matter most. Only for tree models."""
    model = pipe.named_steps["model"]
    if not hasattr(model, "feature_importances_"):
        print(f"  ({name} has no feature_importances_, skipping that plot)")
        return
    names = pipe.named_steps["prep"].get_feature_names_out()
    names = [n.split("__", 1)[-1] for n in names]
    imp = pd.Series(model.feature_importances_, index=names).sort_values()[-15:]

    fig, ax = plt.subplots(figsize=(7, 6))
    imp.plot(kind="barh", ax=ax, color="#3f7cac")
    ax.set_xlabel("Importance")
    ax.set_title(f"Top features according to {name}")
    path = config.FIGURES_DIR / "10_feature_importance.png"
    fig.savefig(path, dpi=120, bbox_inches="tight")
    plt.close(fig)
    print("  saved", path.name)

    imp.sort_values(ascending=False).rename("importance").to_csv(
        config.RESULTS_DIR / "feature_importance.csv"
    )


def leave_one_material_out(df, X, y, best_name):
    """
    Extra check that was not in our original plan, but we added it
    after looking at the EDA.

    Almost every toxic sample in this dataset is CuO or ZnO. A lot of
    the descriptors (enthalpy, oxidation state, molecular weight ...)
    have exactly the same value for every row of a material. So with a
    normal random split, rows of the same material end up in both the
    train and the test set, and the model can partly memorise the
    material instead of learning the chemistry.

    Here we train on 4 materials and test on the 5th one. The scores
    are much lower, and that is the honest estimate of how the model
    would do on a nanoparticle it has never seen.
    """
    print("\n" + "=" * 60)
    print("EXTRA CHECK : LEAVE-ONE-MATERIAL-OUT")
```

```
    print("=" * 60)

    groups = df["NPs"]
    logo = LeaveOneGroupOut()
    rows = []

    for train_idx, test_idx in logo.split(X, y, groups):
        material = groups.iloc[test_idx].iloc[0]
        y_tr = y.iloc[train_idx]
        y_te = y.iloc[test_idx]

        if y_tr.nunique() < 2:
            # Cannot train a classifier on a single class.
            rows.append(
                {
                    "Held_out_material": material,
                    "n_samples": len(test_idx),
                    "n_toxic": int(y_te.sum()),
                    "Accuracy": np.nan,
                    "Recall_toxic": np.nan,
                }
            )
            continue

        pipe = make_pipeline(get_models()[best_name])
        pipe.fit(X.iloc[train_idx], y_tr)
        pred = pipe.predict(X.iloc[test_idx])

        rows.append(
            {
                "Held_out_material": material,
                "n_samples": len(test_idx),
                "n_toxic": int(y_te.sum()),
                "Accuracy": accuracy_score(y_te, pred),
                "Recall_toxic": (
                    recall_score(y_te, pred, zero_division=0)
                    if y_te.sum() > 0
                    else np.nan
                ),
            }
        )

    out = pd.DataFrame(rows)
    print(out.round(3).to_string(index=False))
    print(f"\nMean accuracy over held-out materials: {out['Accuracy'].mean():.3f}")
    print("This is the number we quote as the honest generalisation estimate.")
    out.to_csv(config.RESULTS_DIR / "leave_one_material_out.csv", index=False)


def main():
    if not config.CLEAN_FILE.exists():
        raise FileNotFoundError("Run 'python src/preprocess.py' first.")

    df = pd.read_csv(config.CLEAN_FILE)
    feature_cols = config.NUMERIC_FEATURES + config.CATEGORICAL_FEATURES
    X = df[feature_cols]
    y = df[config.TARGET_NUMERIC]

    config.FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    config.MODEL_DIR.mkdir(parents=True, exist_ok=True)
    config.RESULTS_DIR.mkdir(parents=True, exist_ok=True)

    # Train / test split, stratified so both sets keep the same
    # toxic to non-toxic ratio.
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=config.TEST_SIZE,
        random_state=config.RANDOM_STATE,
        stratify=y,
    )
    print(f"Train: {len(X_train)} rows    Test: {len(X_test)} rows")
    print(f"Toxic in train: {y_train.sum()}    Toxic in test: {y_test.sum()}")

    print("\n" + "=" * 60)
```

```python
    print("STEPS 8-11 : TRAIN, TEST AND EVALUATE THE MODELS")
    print("=" * 60)

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=config.RANDOM_STATE)
    fitted = {}
    scores = []

    for name, model in get_models().items():
        pipe = make_pipeline(model)
        pipe.fit(X_train, y_train)
        fitted[name] = pipe

        row = evaluate(name, pipe, X_test, y_test)
        cv_f1 = cross_val_score(
            make_pipeline(model), X_train, y_train, cv=cv, scoring="f1"
        )
        row["CV_F1_mean"] = cv_f1.mean()
        row["CV_F1_std"] = cv_f1.std()
        scores.append(row)

        print(f"\n--- {name} ---")
        print(
            classification_report(
                y_test,
                pipe.predict(X_test),
                target_names=["Non-Toxic", "Toxic"],
                zero_division=0,
            )
        )

    table = (
        pd.DataFrame(scores)
        .sort_values("F1", ascending=False)
        .reset_index(drop=True)
    )

    print("=" * 60)
    print("STEP 11 : PERFORMANCE COMPARISON")
    print("=" * 60)
    print(table.round(3).to_string(index=False))
    table.to_csv(config.METRICS_FILE, index=False)

    print("\nMaking plots ...")
    plot_confusion_matrices(fitted, X_test, y_test)
    plot_roc(fitted, X_test, y_test)
    plot_scores(table)

    # Step 12 - pick the best model using F1, not accuracy, because
    # of the class imbalance.
    best_name = table.iloc[0]["Model"]
    print("\n" + "=" * 60)
    print(f"STEP 12 : BEST MODEL = {best_name}")
    print(
        f"  F1 = {table.iloc[0]['F1']:.3f}    "
        f"ROC-AUC = {table.iloc[0]['ROC_AUC']:.3f}"
    )
    print("=" * 60)

    plot_feature_importance(fitted[best_name], best_name)

    # Retrain the winner on the whole dataset before saving, so the
    # app in step 14 uses every sample we have.
    final = make_pipeline(get_models()[best_name])
    final.fit(X, y)
    joblib.dump(
        {
            "pipeline": final,
            "model_name": best_name,
            "numeric_features": config.NUMERIC_FEATURES,
            "categorical_features": config.CATEGORICAL_FEATURES,
            "feature_order": feature_cols,
        },
        config.BEST_MODEL_FILE,
    )
    print(f"\nSaved best model to {config.BEST_MODEL_FILE}")
```

```
365
366     leave_one_material_out(df, X, y, best_name)
367
368
369 if __name__ == "__main__":
370     main()
```

F Final Prediction System

Step 14

Two front ends. predict.py is the command line tool and holds the shared predict() function; app.py is the Streamlit web app and calls that same function, so the two can never disagree.

src/predict.py — 241 lines

```python
"""
Step 14 - Final Prediction System (command line version)
---------------------------------------------------------
Loads the saved best model and predicts TOXIC / NON-TOXIC for a
nanoparticle described by the user.

The user only gives the 6 properties they can measure. The rest of
the chemical descriptors are looked up from the material name.

Examples:
    python src/predict.py --demo
    python src/predict.py --material ZnO --coresize 40 --hydrosize 300 \
        --surfcharge -12 --surfarea 50 --expotime 24 --dosage 100

Run without arguments to be asked question by question.
"""

import argparse
import sys

import joblib
import pandas as pd

import config


def load_model():
    if not config.BEST_MODEL_FILE.exists():
        print("No trained model found.")
        print("Run 'python src/train_models.py' first.")
        sys.exit(1)
    return joblib.load(config.BEST_MODEL_FILE)


def load_materials():
    if not config.MATERIAL_FILE.exists():
        print("Material lookup missing. Run 'python src/preprocess.py' first.")
        sys.exit(1)
    return pd.read_csv(config.MATERIAL_FILE, index_col="NPs")


def training_ranges():
    """
    Min and max of each input property in the training data.

    We use this to warn the user when they type something far outside
    what the model has ever seen, because the prediction there is just
    extrapolation and should not be trusted.
    """
    if not config.CLEAN_FILE.exists():
        return None
    df = pd.read_csv(config.CLEAN_FILE)
    return df[config.USER_INPUT_FEATURES].agg(["min", "max"])


def check_ranges(values, ranges):
    """Return a list of warning strings for out-of-range inputs."""
    if ranges is None:
        return []
    warnings = []
    for name, value in values.items():
        if name not in ranges.columns:
            continue
        low, high = ranges.loc["min", name], ranges.loc["max", name]
```

```
        if value < low or value > high:
            warnings.append(
                f"{name} = {value} is outside the training range "
                f"({low:g} to {high:g})"
            )
    return warnings


def build_row(material, values, celltype, materials):
    """Put the user values plus the material descriptors into one row."""
    if material not in materials.index:
        raise ValueError(
            f"Unknown material '{material}'. "
            f"Known materials: {', '.join(materials.index)}"
        )
    if celltype not in ("Cancer", "Normal"):
        raise ValueError(f"Cell type must be 'Cancer' or 'Normal', got '{celltype}'")

    missing = [c for c in config.USER_INPUT_FEATURES if c not in values]
    if missing:
        raise ValueError(f"Missing properties: {', '.join(missing)}")

    row = dict(values)
    row.update(materials.loc[material].to_dict())
    row["Celltype"] = celltype
    return pd.DataFrame([row])


def predict(bundle, materials, material, values, celltype):
    row = build_row(material, values, celltype, materials)
    row = row[bundle["feature_order"]]  # same column order as training

    pipeline = bundle["pipeline"]
    label = int(pipeline.predict(row)[0])
    prob = float(pipeline.predict_proba(row)[0][1])
    return label, prob


def show_result(material, values, celltype, label, prob, model_name):
    print()
    print("=" * 50)
    print("        NANOPARTICLE TOXICITY PREDICTION")
    print("=" * 50)
    print(f"  Material          : {material}")
    print(f"  Core size         : {values['coresize']} nm")
    print(f"  Hydrodynamic size : {values['hydrosize']} nm")
    print(f"  Surface charge    : {values['surfcharge']} mV")
    print(f"  Surface area      : {values['surfarea']} m2/g")
    print(f"  Exposure time     : {values['Expotime']} hours")
    print(f"  Dosage            : {values['dosage']} ug/mL")
    print(f"  Cell type         : {celltype}")
    print("-" * 50)
    verdict = "TOXIC" if label == 1 else "NON-TOXIC"
    print(f"  PREDICTION        : {verdict}")
    print(f"  Probability toxic : {prob * 100:.1f}%")
    print(f"  Model used        : {model_name}")
    print("=" * 50)
    for warning in check_ranges(values, training_ranges()):
        print(f"  ! {warning}")
    print("  Note: this is a screening tool, not a lab result.")
    print()


def ask_interactively(materials):
    """Simple question-answer mode for when no arguments are given."""
    print("Enter the nanoparticle details (press Ctrl+C to quit).\n")
    print("Available materials:", ", ".join(materials.index))

    material = input("Material: ").strip()
    while material not in materials.index:
        print("  Not in the list, try again.")
        material = input("Material: ").strip()

    questions = [
        ("coresize", "Core size (nm)"),
```

```
        ("hydrosize", "Hydrodynamic size (nm)"),
        ("surfcharge", "Surface charge / zeta potential (mV)"),
        ("surfarea", "Surface area (m2/g)"),
        ("Expotime", "Exposure time (hours)"),
        ("dosage", "Dosage (ug/mL)"),
    ]
    values = {}
    for key, question in questions:
        while True:
            try:
                values[key] = float(input(f"{question}: ").strip())
                break
            except ValueError:
                print("  Please type a number.")

    celltype = input("Cell type [Cancer/Normal] (default Cancer): ").strip()
    if celltype.lower().startswith("n"):
        celltype = "Normal"
    else:
        celltype = "Cancer"

    return material, values, celltype


def run_demo(bundle, materials):
    """A few ready made examples so we can show the app quickly."""
    examples = [
        ("ZnO", dict(coresize=40, hydrosize=300, surfcharge=-12,
                     surfarea=50, Expotime=24, dosage=100), "Cancer"),
        ("TiO2", dict(coresize=25, hydrosize=200, surfcharge=15,
                      surfarea=90, Expotime=24, dosage=10), "Normal"),
        ("Al2O3", dict(coresize=39.7, hydrosize=267, surfcharge=36.3,
                       surfarea=64.7, Expotime=24, dosage=0.01), "Normal"),
        ("CuO", dict(coresize=30, hydrosize=400, surfcharge=-20,
                     surfarea=30, Expotime=48, dosage=200), "Cancer"),
    ]
    for material, values, celltype in examples:
        label, prob = predict(bundle, materials, material, values, celltype)
        show_result(material, values, celltype, label, prob, bundle["model_name"])


def main():
    parser = argparse.ArgumentParser(
        description="Predict whether a nanoparticle is toxic or not."
    )
    parser.add_argument("--material", help="e.g. ZnO, CuO, TiO2, Al2O3, Fe2O3")
    parser.add_argument("--coresize", type=float, help="core size in nm")
    parser.add_argument("--hydrosize", type=float, help="hydrodynamic size in nm")
    parser.add_argument("--surfcharge", type=float, help="surface charge in mV")
    parser.add_argument("--surfarea", type=float, help="surface area in m2/g")
    parser.add_argument("--expotime", type=float, help="exposure time in hours")
    parser.add_argument("--dosage", type=float, help="dosage in ug/mL")
    parser.add_argument("--celltype", default="Cancer", choices=["Cancer", "Normal"])
    parser.add_argument("--demo", action="store_true", help="run some examples")
    args = parser.parse_args()

    bundle = load_model()
    materials = load_materials()

    if args.demo:
        run_demo(bundle, materials)
        return

    needed = ["material", "coresize", "hydrosize", "surfcharge",
              "surfarea", "expotime", "dosage"]
    given = [n for n in needed if getattr(args, n) is not None]

    if not given:
        material, values, celltype = ask_interactively(materials)
    elif len(given) < len(needed):
        missing = [n for n in needed if n not in given]
        print("Missing values:", ", ".join(missing))
        print("Give all of them, or give none to use the interactive mode.")
        sys.exit(1)
    else:
```

```
        material = args.material
        celltype = args.celltype
        values = {
            "coresize": args.coresize,
            "hydrosize": args.hydrosize,
            "surfcharge": args.surfcharge,
            "surfarea": args.surfarea,
            "Expotime": args.expotime,
            "dosage": args.dosage,
        }

    try:
        label, prob = predict(bundle, materials, material, values, celltype)
    except ValueError as err:
        print("Error:", err)
        sys.exit(1)

    show_result(material, values, celltype, label, prob, bundle["model_name"])


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nCancelled.")
        sys.exit(0)
```

app.py — 200 lines

```
"""
Step 14 - Final Prediction System (web version)
------------------------------------------------
A small Streamlit app so we can demo the project without touching
the terminal.

Run:  streamlit run app.py
"""

import sys
from pathlib import Path

import joblib
import pandas as pd
import streamlit as st

# Let this file import the modules inside src/
sys.path.insert(0, str(Path(__file__).resolve().parent / "src"))

import config  # noqa: E402
from predict import check_ranges, training_ranges  # noqa: E402
from predict import predict as predict_toxicity  # noqa: E402

st.set_page_config(page_title="Nanoparticle Toxicity Prediction", page_icon="*")


@st.cache_resource
def load_model():
    if not config.BEST_MODEL_FILE.exists():
        return None
    return joblib.load(config.BEST_MODEL_FILE)


@st.cache_data
def load_materials():
    if not config.MATERIAL_FILE.exists():
        return None
    return pd.read_csv(config.MATERIAL_FILE, index_col="NPs")


@st.cache_data
def load_metrics():
    if not config.METRICS_FILE.exists():
        return None
    return pd.read_csv(config.METRICS_FILE)
```

```python


st.title("Nanoparticle Toxicity Prediction")
st.caption("Mini project - predicting nanoparticle toxicity using machine learning")

bundle = load_model()
materials = load_materials()

if bundle is None or materials is None:
    st.error(
        "The model or the material table is missing.\n\n"
        "Please run these first:\n\n"
        "```\n"
        "python src/download_data.py\n"
        "python src/preprocess.py\n"
        "python src/train_models.py\n"
        "```"
    )
    st.stop()

tab_predict, tab_results, tab_about = st.tabs(
    ["Prediction", "Model results", "About"]
)

# ---------------------------------------------------------------------
# Prediction tab
# ---------------------------------------------------------------------
with tab_predict:
    st.subheader("Enter the nanoparticle properties")

    col1, col2 = st.columns(2)

    with col1:
        material = st.selectbox("Nanoparticle material", list(materials.index))
        coresize = st.number_input(
            "Core size (nm)", min_value=1.0, max_value=500.0, value=40.0, step=1.0
        )
        hydrosize = st.number_input(
            "Hydrodynamic size (nm)",
            min_value=1.0, max_value=3000.0, value=300.0, step=10.0,
        )
        surfcharge = st.number_input(
            "Surface charge / zeta potential (mV)",
            min_value=-100.0, max_value=100.0, value=-12.0, step=1.0,
        )

    with col2:
        surfarea = st.number_input(
            "Surface area (m2/g)",
            min_value=0.1, max_value=500.0, value=50.0, step=1.0,
        )
        expotime = st.number_input(
            "Exposure time (hours)",
            min_value=1.0, max_value=200.0, value=24.0, step=1.0,
        )
        dosage = st.number_input(
            "Dosage (ug/mL)",
            min_value=0.0, max_value=1000.0, value=100.0, step=1.0,
        )
        celltype = st.selectbox("Cell type", ["Cancer", "Normal"])

    if st.button("Predict toxicity", type="primary"):
        values = {
            "coresize": coresize,
            "hydrosize": hydrosize,
            "surfcharge": surfcharge,
            "surfarea": surfarea,
            "Expotime": expotime,
            "dosage": dosage,
        }
        # Same function the command line version uses, so the two can
        # never drift apart and give different answers.
        label, prob = predict_toxicity(
            bundle, materials, material, values, celltype
        )
```

```
        st.divider()
        if label == 1:
            st.error(f"Prediction: **TOXIC**  (probability {prob * 100:.1f}%)")
        else:
            st.success(f"Prediction: **NON-TOXIC**  (probability of toxic {prob * 100:.1f}%)")

        st.progress(min(max(prob, 0.0), 1.0))
        st.caption(f"Model used: {bundle['model_name']}")

        for warning in check_ranges(values, training_ranges()):
            st.warning(warning)

        st.info(
            "This is a screening tool built on a small dataset. "
            "It is not a replacement for an actual toxicity assay."
        )

# ---------------------------------------------------------------------
# Results tab
# ---------------------------------------------------------------------
with tab_results:
    st.subheader("How the five models compared")
    metrics = load_metrics()
    if metrics is None:
        st.warning("Run 'python src/train_models.py' to generate the results.")
    else:
        st.dataframe(metrics.round(3), width="stretch")
        st.caption(
            "The best model is chosen by F1 score, not accuracy, because only "
            "about 16% of the samples in the dataset are toxic."
        )

    figures = [
        ("01_class_balance.png", "Class balance"),
        ("03_toxic_vs_nontoxic.png", "Toxic vs non-toxic properties"),
        ("05_correlation_heatmap.png", "Correlation heatmap"),
        ("07_confusion_matrices.png", "Confusion matrices"),
        ("08_roc_curves.png", "ROC curves"),
        ("09_model_comparison.png", "Model comparison"),
        ("10_feature_importance.png", "Feature importance"),
    ]
    for filename, caption in figures:
        path = config.FIGURES_DIR / filename
        if path.exists():
            st.image(str(path), caption=caption, width="stretch")

# ---------------------------------------------------------------------
# About tab
# ---------------------------------------------------------------------
with tab_about:
    st.markdown(
        """
### Nanoparticle Toxicity Prediction Using Data-Driven Methods

Nanoparticles are widely used in drug delivery because they are small,
have a large surface area and can target specific cells. The same
properties can also make them toxic, so being able to predict toxicity
early helps in designing safer particles.

This project takes physicochemical properties of a nanoparticle
(core size, hydrodynamic size, surface charge, surface area, dosage and
exposure time) plus standard chemical descriptors of the material, and
classifies the particle as **toxic** or **non-toxic**.

**Models compared:** Logistic Regression, Decision Tree, K-Nearest
Neighbors, Support Vector Machine and Random Forest.

**Dataset:** the NanoTox metal-oxide nanoparticle dataset
(477 experiments after cleaning, 5 materials: Al2O3, CuO, Fe2O3,
TiO2, ZnO).

**Known limitation:** almost all the toxic samples in the dataset are
CuO and ZnO. When we hold out a whole material and test on it, the
scores drop a lot. So the model works well for materials similar to
```

```
196     the ones it has seen, and should not be trusted for a completely new
197     material. This is discussed in the README.
198             """
199         )
```

Pipeline Runner

Runs steps 3 to 13 in order

> One command that executes the whole pipeline from start to finish.

run_all.py — 40 lines

```python
"""
Runs the whole pipeline in order:
    download -> preprocess -> EDA -> train

Run:  python run_all.py
Then: streamlit run app.py
"""

import subprocess
import sys
from pathlib import Path

SRC = Path(__file__).resolve().parent / "src"

STEPS = [
    ("Step 3  - Dataset collection", "download_data.py"),
    ("Steps 4-5 - Understanding and preprocessing", "preprocess.py"),
    ("Steps 6-7 - EDA and feature analysis", "eda.py"),
    ("Steps 8-12 - Model training and evaluation", "train_models.py"),
]


def main():
    for title, script in STEPS:
        print("\n" + "#" * 70)
        print("#", title)
        print("#" * 70 + "\n")
        result = subprocess.run([sys.executable, script], cwd=SRC)
        if result.returncode != 0:
            print(f"\n{script} failed. Stopping here.")
            sys.exit(result.returncode)

    print("\n" + "#" * 70)
    print("# Done. Now run:  streamlit run app.py")
    print("#" * 70)


if __name__ == "__main__":
    main()
```

Bug Sweep Test Suite

Step 15

The 18 tests written during the bug sweep. The leakage test and the duplicate test are the two that caught real problems.

tests/test_pipeline.py — 226 lines

```python
"""
Basic tests for the project.

We wrote these while doing the bug sweep at the end of the project.
They check the things that would silently give us wrong results.

Run:  pytest -v
(from the project folder, after running python run_all.py)
"""

import sys
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import pytest

PROJECT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT / "src"))

import config  # noqa: E402
import predict as predict_module  # noqa: E402
from preprocess import clean_data, load_raw  # noqa: E402


# ---------------------------------------------------------------------
# Fixtures
# ---------------------------------------------------------------------
@pytest.fixture(scope="module")
def clean():
    if not config.CLEAN_FILE.exists():
        pytest.skip("Run 'python run_all.py' first.")
    return pd.read_csv(config.CLEAN_FILE)


@pytest.fixture(scope="module")
def bundle():
    if not config.BEST_MODEL_FILE.exists():
        pytest.skip("Run 'python run_all.py' first.")
    return joblib.load(config.BEST_MODEL_FILE)


@pytest.fixture(scope="module")
def materials():
    if not config.MATERIAL_FILE.exists():
        pytest.skip("Run 'python run_all.py' first.")
    return pd.read_csv(config.MATERIAL_FILE, index_col="NPs")


# ---------------------------------------------------------------------
# Data tests
# ---------------------------------------------------------------------
def test_raw_file_exists():
    assert config.RAW_FILE.exists(), "raw dataset missing, run download_data.py"


def test_no_leaky_column_in_clean_data(clean):
    """
    The most important test. 'viability' is what the toxic label was
    made from, so if it ever comes back into the clean file our
    accuracy becomes fake.
    """
    for col in config.LEAKY_COLS:
        assert col not in clean.columns, f"leaky column '{col}' is back in the data"
```

```python


def test_target_is_binary(clean):
    assert set(clean[config.TARGET_NUMERIC].unique()) == {0, 1}


def test_no_missing_values(clean):
    assert clean.isna().sum().sum() == 0


def test_no_duplicate_rows(clean):
    assert clean.duplicated().sum() == 0


def test_all_features_present(clean):
    for col in config.NUMERIC_FEATURES + config.CATEGORICAL_FEATURES:
        assert col in clean.columns, f"feature '{col}' missing from clean data"


def test_feature_lists_do_not_overlap():
    """A column must not be both a user input and a material constant."""
    overlap = set(config.USER_INPUT_FEATURES) & set(config.MATERIAL_FEATURES)
    assert not overlap, f"columns in both lists: {overlap}"
    assert (
        set(config.USER_INPUT_FEATURES) | set(config.MATERIAL_FEATURES)
    ) == set(config.NUMERIC_FEATURES)


def test_material_columns_really_are_constant(clean):
    """
    The app fills these in from the material name. If any of them
    actually varied within a material, the app would be feeding the
    model a wrong value.
    """
    for col in config.MATERIAL_FEATURES:
        counts = clean.groupby("NPs")[col].nunique()
        bad = counts[counts > 1]
        assert bad.empty, f"'{col}' is not constant for: {list(bad.index)}"


def test_preprocessing_is_repeatable():
    """Running clean_data twice on the same raw data gives the same thing."""
    a = clean_data(load_raw())
    b = clean_data(load_raw())
    pd.testing.assert_frame_equal(a, b)


# ---------------------------------------------------------------------
# Model tests
# ---------------------------------------------------------------------
def test_model_bundle_has_what_we_need(bundle):
    for key in ["pipeline", "model_name", "feature_order"]:
        assert key in bundle


def test_feature_order_matches_config(bundle):
    expected = config.NUMERIC_FEATURES + config.CATEGORICAL_FEATURES
    assert bundle["feature_order"] == expected


def test_prediction_returns_valid_output(bundle, materials):
    values = dict(
        coresize=40, hydrosize=300, surfcharge=-12,
        surfarea=50, Expotime=24, dosage=100,
    )
    label, prob = predict_module.predict(
        bundle, materials, "ZnO", values, "Cancer"
    )
    assert label in (0, 1)
    assert 0.0 <= prob <= 1.0


def test_unknown_material_is_rejected(bundle, materials):
    values = dict(
        coresize=40, hydrosize=300, surfcharge=-12,
```

```
        surfarea=50, Expotime=24, dosage=100,
    )
    with pytest.raises(ValueError):
        predict_module.predict(bundle, materials, "Gold", values, "Cancer")


def test_column_order_does_not_change_the_answer(bundle, materials):
    """
    We shuffle the dict order. build_row must still put the columns in
    the training order, otherwise the model reads the wrong numbers.
    """
    ordered = dict(
        coresize=40, hydrosize=300, surfcharge=-12,
        surfarea=50, Expotime=24, dosage=100,
    )
    shuffled = dict(reversed(list(ordered.items())))
    a = predict_module.predict(bundle, materials, "CuO", ordered, "Cancer")
    b = predict_module.predict(bundle, materials, "CuO", shuffled, "Cancer")
    assert a == b


def test_known_toxic_and_nontoxic_cases(bundle, materials):
    """
    Sanity check with chemistry we know: CuO and ZnO are the toxic ones
    in this dataset, Al2O3 and Fe2O3 are not.
    """
    values = dict(
        coresize=40, hydrosize=300, surfcharge=-12,
        surfarea=50, Expotime=24, dosage=100,
    )
    toxic_label, _ = predict_module.predict(
        bundle, materials, "CuO", values, "Cancer"
    )
    safe_label, _ = predict_module.predict(
        bundle, materials, "Al2O3", values, "Normal"
    )
    assert toxic_label == 1, "CuO at a high dose should come out toxic"
    assert safe_label == 0, "Al2O3 should come out non-toxic"


def test_model_beats_the_lazy_baseline():
    """
    A model that always says 'non-toxic' would score about 0.84
    accuracy and 0.0 F1. Ours has to do better than that.
    """
    if not config.METRICS_FILE.exists():
        pytest.skip("Run 'python src/train_models.py' first.")
    table = pd.read_csv(config.METRICS_FILE)
    best = table.sort_values("F1", ascending=False).iloc[0]
    assert best["F1"] > 0.5, "best model is no better than guessing the majority"
    assert best["Recall"] > 0.5, "model misses more than half of the toxic particles"


def test_app_and_cli_use_the_same_function():
    """
    We had a bug where app.py had its own copy of the prediction code
    and could drift from predict.py. This test makes sure the app
    imports the shared one.
    """
    source = (PROJECT / "app.py").read_text(encoding="utf-8")
    assert "from predict import predict" in source
    assert "predict_proba" not in source, (
        "app.py is doing its own prediction again instead of calling predict.py"
    )


def test_probability_and_label_agree(bundle, materials):
    """label 1 should mean probability >= 0.5 (default threshold)."""
    rng = np.random.default_rng(0)
    for material in materials.index:
        for _ in range(5):
            values = dict(
                coresize=float(rng.uniform(5, 120)),
                hydrosize=float(rng.uniform(50, 1800)),
                surfcharge=float(rng.uniform(-50, 50)),
```

```
                surfarea=float(rng.uniform(1, 200)),
                Expotime=float(rng.choice([6, 12, 24, 48, 72])),
                dosage=float(rng.uniform(0.001, 300)),
            )
            label, prob = predict_module.predict(
                bundle, materials, material, values, "Cancer"
            )
            assert label == int(prob >= 0.5), (
                f"{material}: label {label} but probability {prob}"
            )
```